

Everything You Need to Know to Do the Four Tasks

A plain-language explanation of every statistical and machine-learning idea the Day 1 hands-on tasks depend on — with a picture for each one. No prior statistics required.

Who this is for. If a phrase in the task pack — *bootstrap interval*, *precision at the deployment prior*, *McNemar's test*, *C_p*, *kappa* — is unfamiliar, this document explains it before you need it. If all of them are familiar, use the cheat sheet on the last two pages and skip the rest. Nothing here assumes you have studied statistics.

Part	Supports	Concepts covered
Part 0	All four tasks	The five ideas the whole day is built on
Part 1	Task 1 — SPC console	Variation, the normal distribution, subgroups, control charts, control vs specification limits, runs rules, average run length, C_p and C_{pk} , confidence intervals, the bootstrap, cost-based thresholds
Part 2	Task 2 — Operating-point studio	Class imbalance, the confusion matrix, precision and recall, thresholds, ROC versus precision–recall, base rates, label shift, data leakage, feature engineering, alarm budgets
Part 3	Task 3 — Taxonomy console	Taxonomy design, inter-annotator agreement, Cohen's and Fleiss' kappa, the label-quality ceiling, Pareto analysis, cost weighting, slicing, headroom, backlog scoring
Part 4	Task 4 — Readiness review	Hypotheses, p-values, paired versus unpaired designs, McNemar's test, permutation tests, Type I and II errors, power, effect size, multiple comparisons, peeking, calibration, drift
Appendix	All four tasks	One-page formula sheet and a glossary

How to read a section. Each concept has the same three parts: **what it is** in one or two sentences, **a picture**, and **why the task needs it**. If you are short of time, read the pictures and their captions — they carry most of the content on their own.

1 · Everything varies, and some of that variation means nothing

Measure the same machined part on the same gauge twice and you get two different numbers. Measure a hundred parts from a perfectly healthy process and they scatter. That scatter is not a problem to be solved — it is what a stable process looks like. The engineering skill is telling the scatter that means nothing apart from the scatter that means something changed.

The same distinction governs every AI system you will build. A model's score moves between evaluation runs. Some of that movement is noise. Some of it is your change working. Confusing the two is the single most common failure in industrial AI, and it is why Day 1 spends so long on measurement before it touches modelling.

2 · A number without an interval is not a measurement

"Accuracy is 0.84" tells you almost nothing on its own. Measured on how many items? If eighty, the truth could reasonably be anywhere from 0.75 to 0.90, and an improvement from 0.81 to 0.84 is indistinguishable from luck. Every headline figure in every task must carry an interval, because the interval is what tells you whether the figure can support the decision you are about to make with it.

3 · Every automatic decision has two ways to be wrong, and they cost different amounts

A system that flags a problem can flag one that is not there, or miss one that is. Stopping a line that was fine costs money. Letting a defective part reach assembly costs more. These two costs are almost never equal, which means the setting that balances them is almost never the middle. Choosing that setting is an economic decision that happens to be expressed in statistics — not the other way round.

4 · The bottleneck is usually the data, not the model

The instinct when a system underperforms is to reach for a bigger model or more data. On a stalled industrial proof of concept, the constraint is nearly always somewhere else: labels that two experts would assign differently, a sensor that does not capture the signal, a feature that quietly contains the answer, or an evaluation set that was never honest to begin with. Tasks 2 and 3 are built so that you find this out by doing rather than by being told.

5 · You can only improve what you can measure honestly

Every shortcut that makes a number look better without making the system better — tuning against the test set, peeking at results until they turn favourable, quietly changing which metric you report — produces a system that performs in the lab and fails in the plant. Each of the four tasks contains one of these shortcuts, deliberately, so that you meet it in a room where it costs nothing.

A note on notation. Three symbols appear throughout and nothing else does. σ (sigma) is the standard deviation — the typical distance of a measurement from the average. μ (mu) is the true average of a process, as opposed to the average of the sample you happened to measure. n is how many things you measured. If you can read those three, you can read everything in this document.

1.1 The normal distribution, and what σ actually means

Take a stable machining process and measure a thousand parts. The heights cluster around a central value and thin out symmetrically on both sides. That shape is the normal distribution, and it turns up whenever many small independent influences add together — which is exactly what a machining process is.

The standard deviation, σ , is the natural unit of that scatter. It is not an arbitrary choice: once you know σ , you know what fraction of parts falls in any band around the centre, and those fractions are fixed.

Figure 1 · What σ means: the spread of a stable process

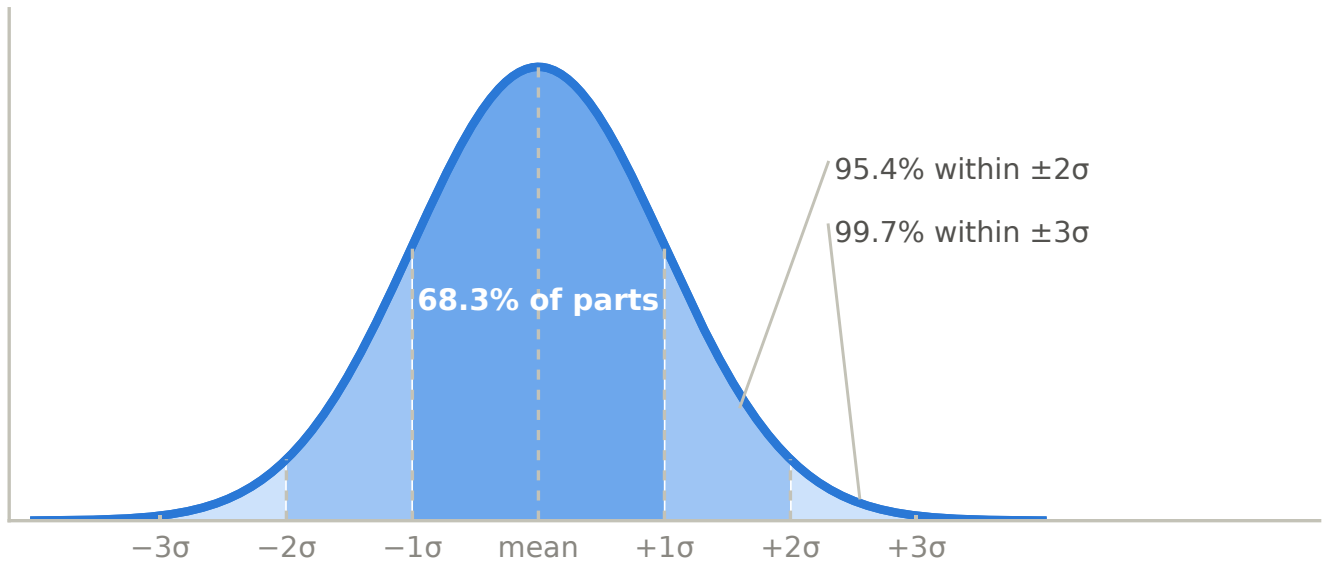


Figure 1. About 68% of parts fall within one σ of the average, 95% within two, and 99.7% within three. This is why control charts use $\pm 3\sigma$: outside that band, one part in roughly 370 lands there by chance alone, so a point out there is worth investigating.

1.2 Common cause and special cause — the distinction the whole task turns on

Common cause variation is the process breathing. Vibration, small temperature changes, gauge repeatability, material batch differences. It is always present, it has no single culprit, and chasing an individual point caused by it wastes a shift. Reducing it means changing the process itself.

Special cause variation is something specific having gone wrong — a tool wearing, a fixture slipping, an operator changing at shift handover. It has a findable cause, and finding it is worth doing.

Figure 2 · Two kinds of variation — and only one is worth investigating

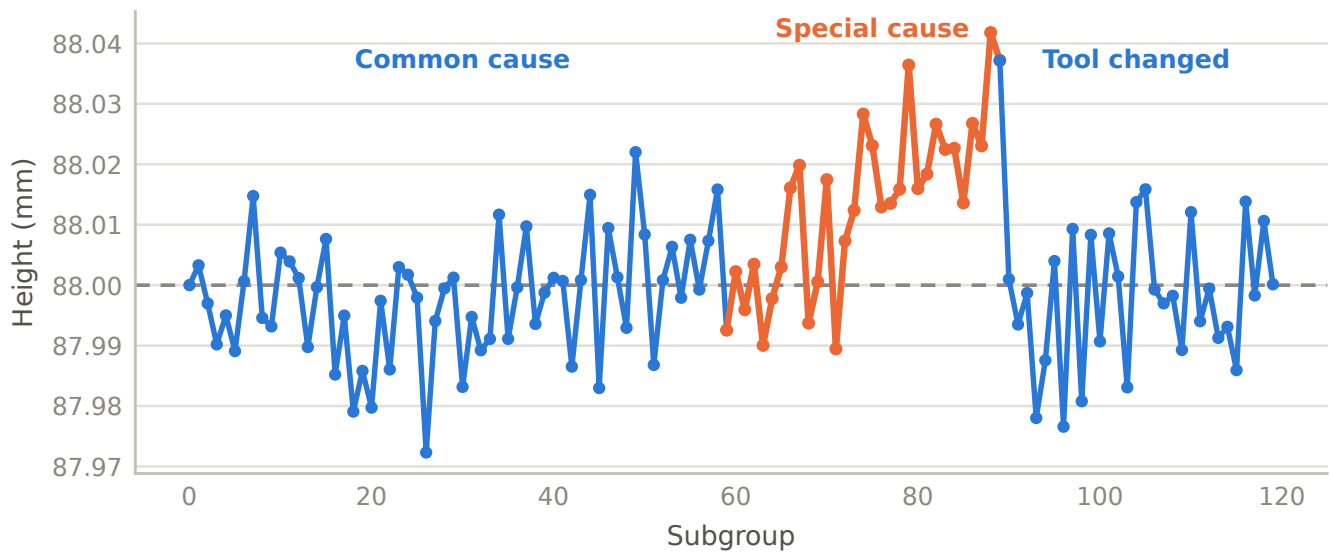


Figure 2. The blue stretches and the orange stretch contain exactly the same kind of measurement. Only the orange one has a cause worth hunting. A control chart exists to make that distinction automatically and consistently, instead of leaving it to whoever is looking at the numbers.

This is the same distinction Module 1 calls aleatoric and epistemic uncertainty. Common cause is *aleatoric* — irreducible noise you must design around. Special cause is *epistemic* — something you do not yet know, which you can find out. Quality engineering has had these two words for seventy years; machine learning renamed them. They are the same idea, and you already understand it.

1.3 Why we measure five parts at a time, not one

Individual parts scatter widely. Averages of several parts scatter much less — and the amount less is exactly predictable. If individual parts have standard deviation σ , then the average of n parts has standard deviation σ divided by the square root of n .

The standard error of a mean

$$\sigma_{\text{mean}} = \sigma / \sqrt{n} \rightarrow \text{for } \sigma = 0.025 \text{ mm and } n = 5: 0.025 / 2.236 = 0.011 \text{ mm}$$

Averaging five parts more than halves the scatter. Averaging twenty would halve it again — the square root is why quadrupling the sample only doubles the precision, and that single fact drives every sample-size argument on Day 1.

Figure 3 · Averaging five parts more than halves the spread

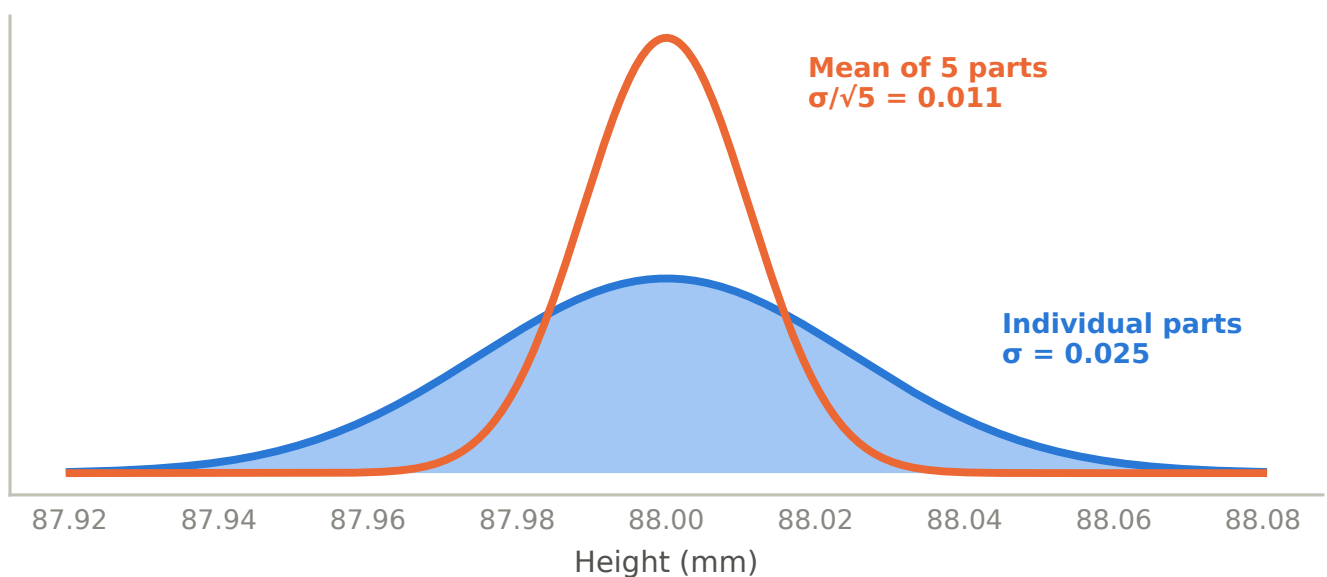


Figure 3. Both curves describe the same process. The narrow one is what you see when you plot the average of each group of five, which is why a chart of subgroup means detects a small shift that a chart of individual parts would hide.

1.4 Anatomy of a control chart

A control chart plots the subgroup average over time, with three horizontal reference lines: a centre line at the process average, and upper and lower control limits at three standard errors either side. Points inside the band are the process behaving. Points outside it are a signal.

Figure 4 · Anatomy of an X-bar control chart

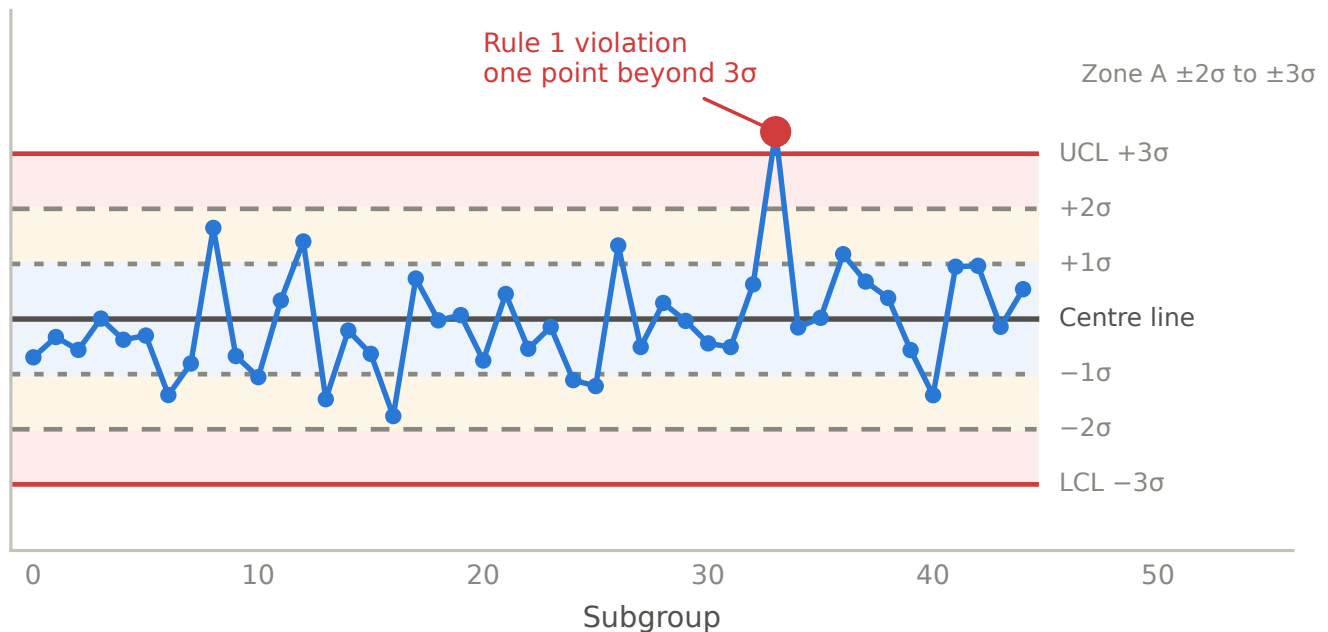


Figure 4. The band is divided into zones at one and two σ , because several of the detection rules refer to them. The $\bar{X}$ chart tracks the average — where the process is centred. A second chart, the R chart, tracks the range within each subgroup — how spread out the process is. You need both: a process can drift off centre without changing spread, or become erratic without moving off centre.

1.5 Control limits are not specification limits

This confusion is the most common error in industrial SPC, and it has an exact analogue in AI work, so it is worth being precise about.

	Control limits	Specification limits
Come from	The process itself — measured from data	The design drawing — set by engineering or the customer
Answer	"Is this process behaving the way it normally does?"	"Will this part be accepted?"
Change when	The process genuinely changes	The requirement changes
In AI terms	The model's normal operating range	The business target you were given

Figure 5 • These two are unrelated. Never plot one as the other.

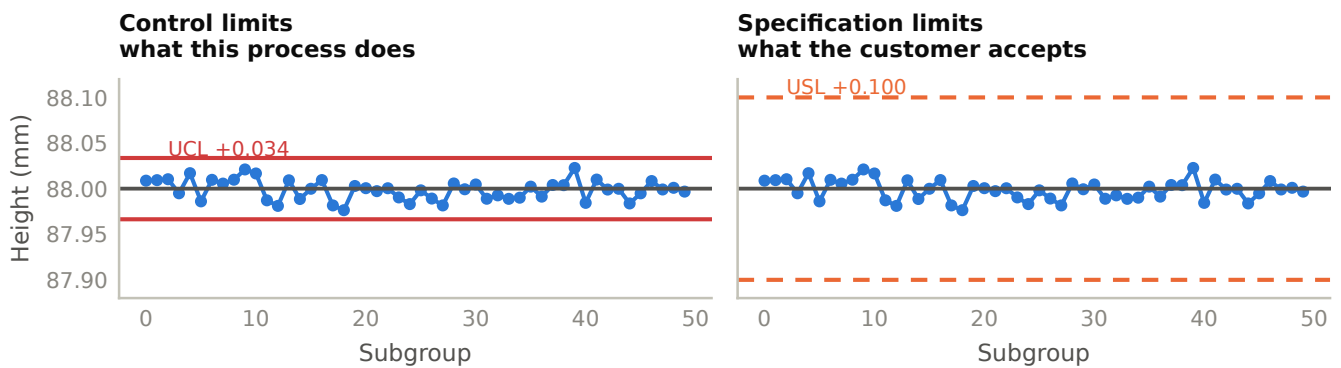


Figure 5. Same data, two different sets of lines, no relationship between them. A process can be perfectly in control and still produce scrap — if its natural spread is wider than the tolerance. It can also be out of control while every part still passes. Plotting one as the other means you will react to the wrong events.

1.6 Where the limits come from — and why this is Task 1's trap

Control limits are estimated from the data. If the data you estimate them from already contains the faults you are trying to detect, the estimate of σ comes out too large, the limits stretch to accommodate the faults, and the chart concludes that everything is fine.

The fix is a discipline, not a formula: estimate the limits from a window you have confirmed was in control, then **freeze them** and judge everything afterwards against those frozen limits.

Figure 6 • Contaminated limits hide the fault they were built to find



Figure 6. Identical data, both panels. On the left the limits come from a clean baseline window and the fault is caught early. On the right they were computed from everything, including the fault — and the fault has widened the limits enough to hide itself.

The same mistake in AI work. Fitting a scaler, an imputer or a threshold on the whole dataset before splitting it does exactly this: the test set has influenced the thing it is meant to judge. It appears as item four on the seven-point leakage checklist, and it is why Module 2A insists that every transformation is fitted inside the training fold only.

1.7 The runs rules — catching things a single point cannot show

A point outside three sigma is an obvious signal, but many real faults never produce one. A tool wearing slowly nudges the average up a few thousandths at a time; a shift handover produces a small step that stays. Both are plainly visible as *patterns* long before any single point leaves the band. The runs rules — eight of them, standardised by Lloyd Nelson in 1984 — are the patterns worth watching for.

Figure 7 · What the runs rules are looking for — all four of these are signals

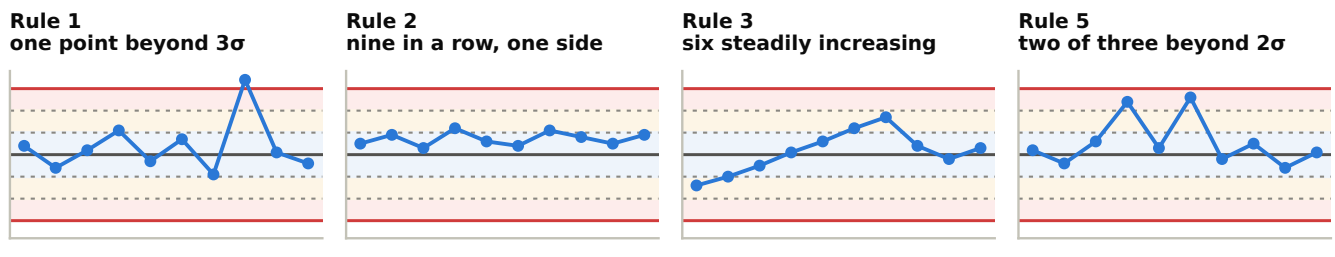


Figure 7. Four of the eight rules. None of the last three panels contains a point outside the control limits, and all three are signals. Rule 2 catches a sustained shift, rule 3 catches a trend such as tool wear, and rule 5 catches a shift too small to break the outer limit but large enough to cluster near it.

Rule	Pattern	What it typically means
1	One point beyond 3σ	Something abrupt — a gross error, a crash, a wrong part
2	Nine points in a row on the same side of the centre	The process has shifted and stayed shifted
3	Six points in a row steadily rising or falling	A trend — tool wear, thermal growth, gradual loosening
4	Fourteen points alternating up and down	Two alternating sources — two spindles, two fixtures, two operators
5	Two of three consecutive points beyond 2σ , same side	A moderate shift, caught sooner than rule 1 would
6	Four of five consecutive points beyond 1σ , same side	A small sustained shift
7	Fifteen points in a row all within 1σ	Suspiciously calm — often a gauge problem or fabricated data
8	Eight points in a row all outside 1σ	Mixture of two distinct populations

1.8 Average run length — the price you pay for sensitivity

Every rule you switch on catches faults sooner and also fires more often when nothing is wrong. ARL_0 is the average number of subgroups between false alarms while the process is healthy — higher is calmer. ARL_1 is the average number of subgroups to detect a real fault — lower is faster. You cannot improve both at once, and choosing between them is the whole of Task 1 Step 5.

Figure 8 · Every rule you add finds faults sooner and cries wolf more often

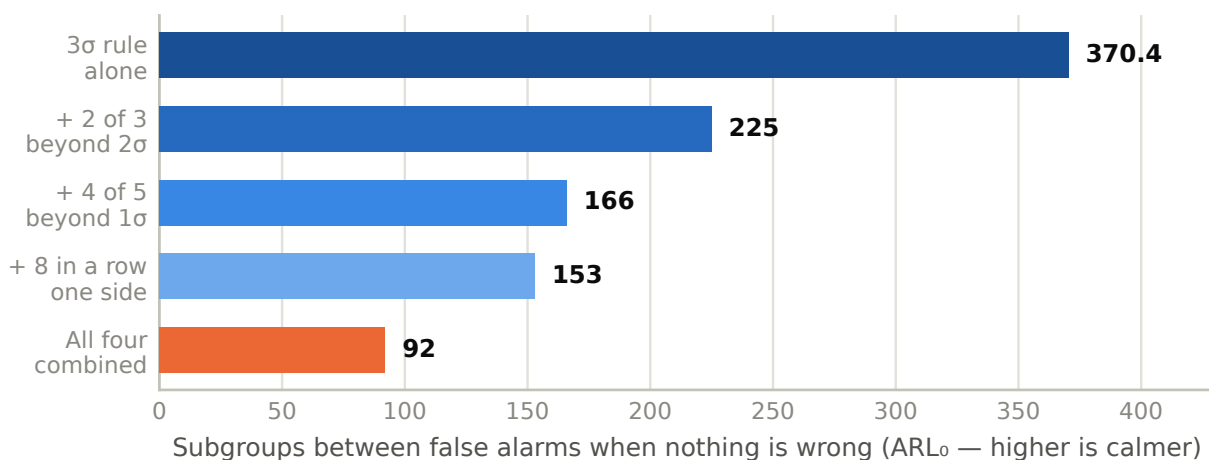


Figure 8. The plain 3σ rule raises a false alarm about once every 370 subgroups. Add the supplementary runs rules and that drops toward 92 — roughly four times as many false alarms, in exchange for catching small shifts far sooner. Neither column is right; the right answer depends on what a false alarm and a missed fault each cost you.

This is precisely the same trade-off as a classifier threshold. ARL_0 is the false-positive rate. ARL_1 is detection latency. Every argument you make in Task 1 about which rules to enable, you will make again in Task 2 about where to set the threshold, and again in Task 4 about whether to ship.

1.9 Capability — C_p and C_{pk}

Control tells you whether the process is stable. **Capability** tells you whether a stable process is good enough for the tolerance it has to meet — two different questions with two different answers.

Capability indices

$C_p = (USL - LSL) / (6\sigma)$ — how the spread compares with the tolerance width $C_{pk} = \min(USL - \mu, \mu - LSL) / (3\sigma)$ — the same, but penalised for being off centre

C_p ignores where the process sits. C_{pk} does not. A process can have an excellent C_p and a poor C_{pk} simply by running off centre — and that is good news, because re-centring is usually far cheaper than reducing variation. Conventional gates are $C_{pk} \geq 1.33$ for ongoing production and ≥ 1.67 for safety-critical characteristics.

Figure 9 • C_p measures spread. C_{pk} measures spread and centring together.

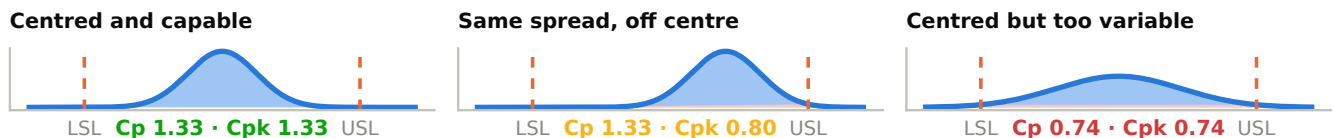


Figure 9. The dashed lines are the tolerance. Left: spread and centring both fine. Middle: identical spread, shifted off centre — C_p unchanged, C_{pk} collapses, and the fix is an offset adjustment. Right: centred but too variable — both indices fall, and the fix is a much bigger job.

1.10 Confidence intervals, and the bootstrap

Every number you compute from a sample would come out slightly differently if you had measured a different sample. A **confidence interval** is the range of values consistent with the data you actually have. A 95% interval means: if you repeated this whole exercise many times, about 95% of the intervals you construct would contain the true value.

For some quantities there is a formula. For most interesting ones — C_{pk} , recall, a difference between two systems, a task success rate — there either is no simple formula or its assumptions do not hold. The **bootstrap** solves all of them the same way, and it is twenty lines of code.

The bootstrap, in five steps

1. Draw n items from your sample of n , with replacement
2. Recompute the metric on that resample
3. Repeat 2,000 to 10,000 times
4. You now have a distribution of the metric
5. The 2.5th and 97.5th percentiles are your 95% interval

"With replacement" is the crucial part: an item can be drawn twice, or not at all. That is what simulates the luck of the draw you were subject to when you collected the sample in the first place.

Figure 10 · The bootstrap: resample, recompute, take the middle 95%

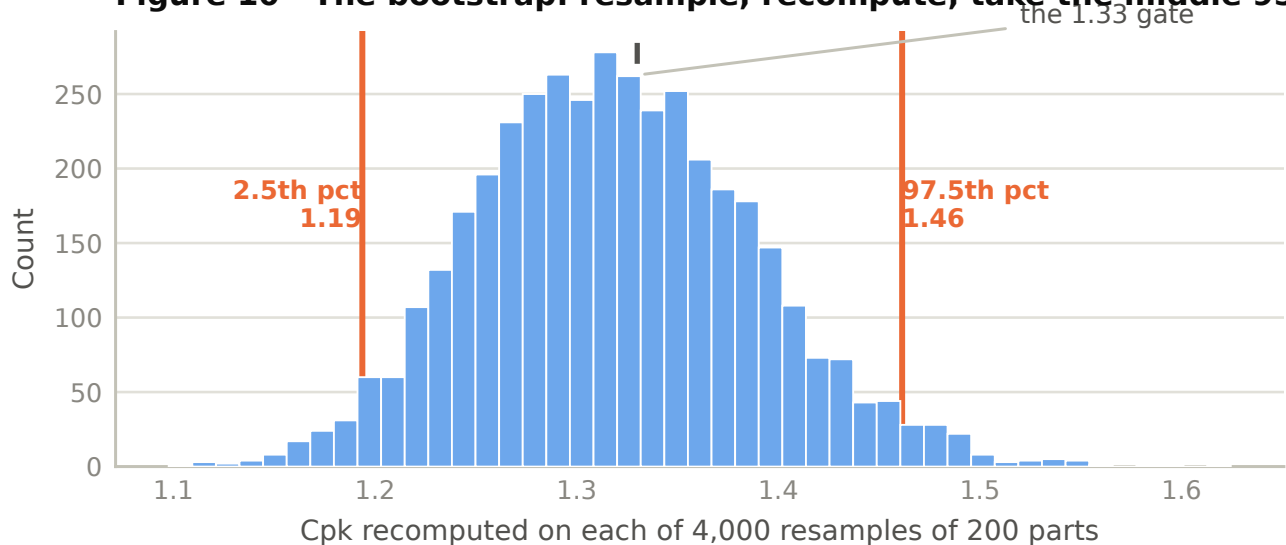


Figure 10. Four thousand resamples of 200 parts, C_{pk} recomputed on each. The spread of this histogram is the uncertainty in your single measured value. The middle 95% runs from 1.19 to 1.46 — which comfortably contains the 1.33 gate, meaning this process cannot be shown to pass it or to fail it.

How wide the interval is depends on how much you measured — and it narrows disappointingly slowly, because of that square root again.

Figure 11 · Why "measure more" is not always the answer

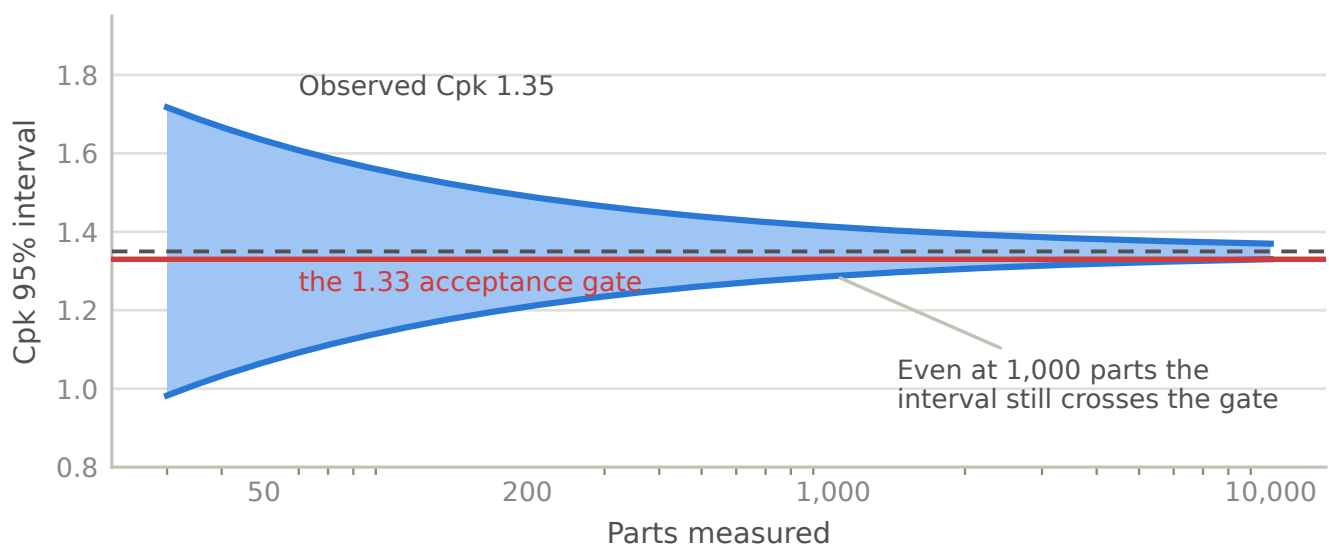


Figure 11. The interval around an observed C_{pk} of 1.35, as the number of parts grows. Even at a thousand parts it still crosses the 1.33 gate. Separating 1.35 from 1.33 with confidence would take on the order of ten thousand parts — so the honest conclusion is not "measure more", it is that a process designed at $C_p = 1.33$ can never be *demonstrated* to clear a 1.33 gate.

Why this matters far beyond Task 1. This is a power argument made with calipers. The identical reasoning in Task 4 tells you whether your evaluation set is large enough to detect the model improvement you are hoping for — and most teams discover theirs is not.

1.11 Choosing the operating point by cost

Once you can measure how often your rules fire falsely and how quickly they catch real faults, the choice of rule set stops being a statistical question and becomes an arithmetic one: attach a price to each outcome and pick the cheapest configuration.

Figure 12 · The threshold is an economic decision, not a statistical one

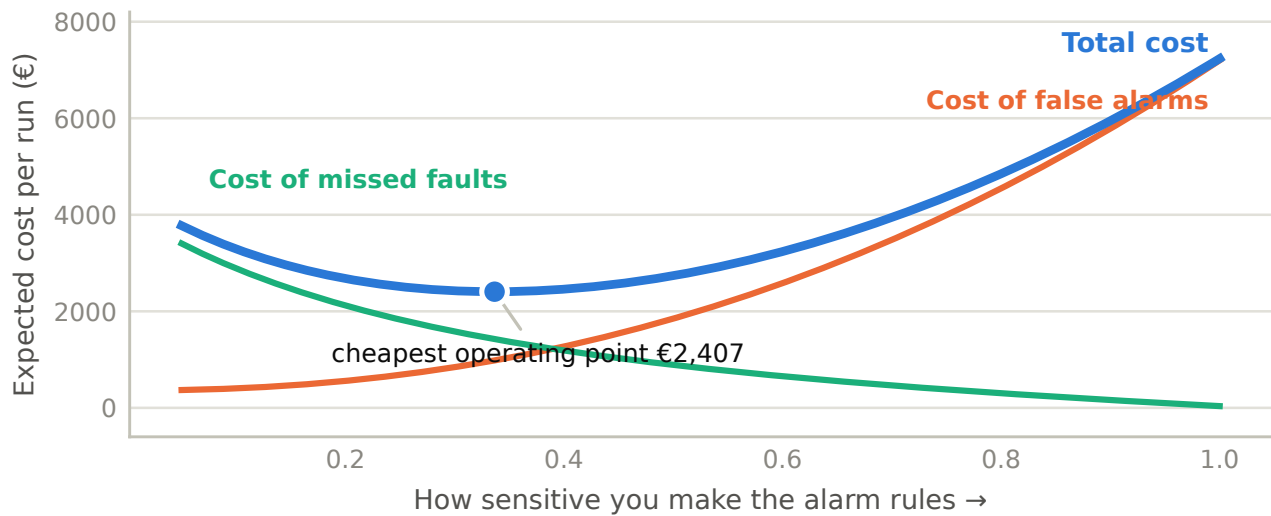


Figure 12. Make the rules more sensitive and the cost of false alarms rises while the cost of missed faults falls. Total cost is the sum, and it has a minimum that is almost never at the setting anyone would have picked by intuition. The same U-shaped curve reappears in Task 2 as the threshold-versus-cost plot.

2.1 Class imbalance, and why accuracy stops meaning anything

A classifier's **accuracy** is the fraction of predictions it gets right. That sounds like the obvious thing to measure, and it is almost useless the moment one outcome is much rarer than the other — which is the situation in every failure-prediction, defect-detection and fraud problem ever built.

Figure 13 · "Always say healthy" scores 96.6% and is worth nothing

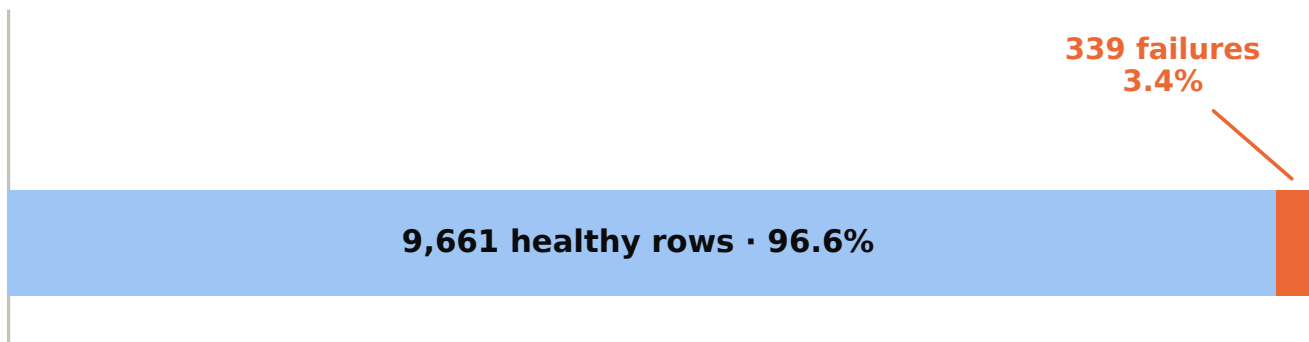


Figure 13. In the Task 2 dataset, 3.4% of rows are failures. A model that ignores every sensor and answers "healthy" every time is right 96.6% of the time. It would also have prevented exactly nothing. Accuracy has rewarded it for the shape of the data rather than for any skill.

The rule to carry out of this section. On imbalanced data, never report overall accuracy. Report performance *on the rare class* — the one you actually care about — and report it with an interval.

2.2 The confusion matrix, precision and recall

Every prediction on a two-outcome problem lands in one of four boxes. Those four counts contain everything there is to know, and every metric worth quoting is a ratio built from them.

Figure 14 · The confusion matrix, and the only two ratios that matter here

	Model says FAILURE	Model says HEALTHY	
Actually failed	True positive TP caught it	False negative FN machine failed, no warning	Recall = $TP / (TP + FN)$ of the failures that happened, how many did we catch?
Actually healthy	False positive FP engineer sent, nothing there	True negative TN correctly quiet	Precision = $TP / (TP + FP)$ of the alerts we raised, how many were real?

Figure 14. Recall answers the question the maintenance manager asks — of the failures that happened, how many did we catch? Precision answers the question the field engineer asks — of the alerts I was sent, how many were real? The two pull in opposite directions, and you cannot maximise both.

The four ratios you will use

Recall (sensitivity) = $TP / (TP + FN)$ caught, out of everything that happened
 Precision = $TP / (TP + FP)$ real, out of everything we flagged
 False positive rate = $FP / (FP + TN)$ how often we

cry wolf on a healthy machine $F1 = 2 \cdot P \cdot R / (P + R)$ the harmonic mean, when you need one number

F1 is convenient and slightly dishonest — it weights precision and recall equally, which is a business decision disguised as a formula. Use it to compare experiments quickly; never use it to justify a deployment.

2.3 One model, many operating points

A classifier does not output a decision. It outputs a score — a number between zero and one. The decision is made by comparing that score with a **threshold**, and the threshold is a separate choice that you make, not something the model learned.

Lower the threshold and you catch more failures and raise more false alarms. Raise it and the reverse. The model is unchanged throughout. This is why threshold tuning is nearly free and frequently the largest single improvement available — it changes no training at all.

0.5 is not a default, it is an accident. It happens to be the midpoint of the output range, and that is its entire justification. On imbalanced data with asymmetric costs it is essentially always the wrong choice. Any threshold you report should be traceable to a cost or a capacity constraint.

2.4 ROC and precision–recall — two pictures of the same model

Rather than pick one threshold, you can plot performance across all of them. There are two standard ways to do it, and on rare-event problems they tell dramatically different stories.

Figure 15 · The same model, two pictures. Only one tells you what using it feels like.

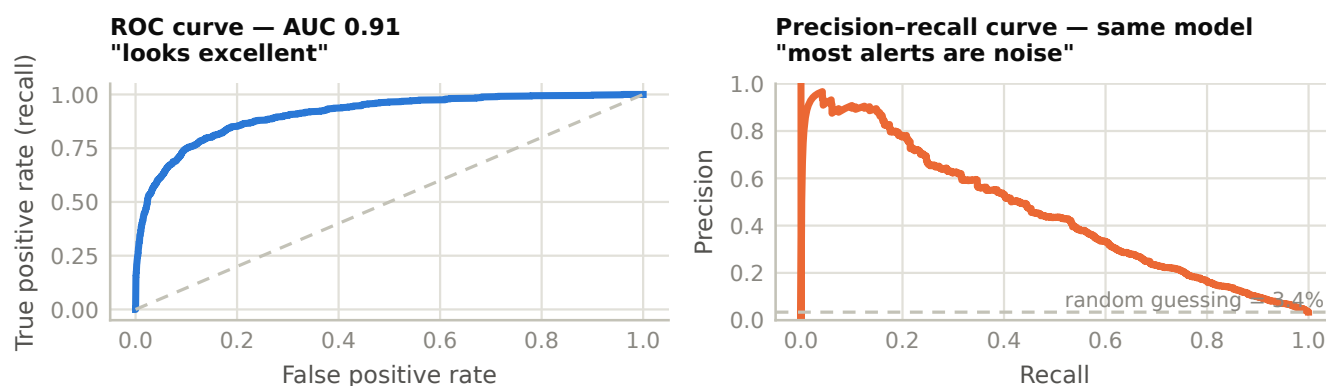


Figure 15. Identical model, identical data. The ROC curve's horizontal axis is the false positive rate, whose denominator is the enormous number of healthy machines — so even a large number of false alarms produces a tiny rate, and the curve looks superb. The precision–recall curve uses the number of predicted positives instead, so every false alarm damages it directly. The right-hand panel is the one that describes what using the system feels like.

Watch for. A high ROC-AUC on a rare-event problem is not evidence of a deployable system. Report the precision–recall curve, and quote the specific operating point you intend to run at.

2.5 Base rate — why precision collapses when the target is rare

Precision does not depend only on the model. It depends just as strongly on how common the thing you are hunting actually is. Hold the model perfectly constant, deploy it somewhere the event is ten times rarer, and precision falls off a cliff.

Precision from sensitivity, false-positive rate and base rate

$$\text{precision} = \frac{\text{TPR} \cdot \pi}{(\text{TPR} \cdot \pi + \text{FPR} \cdot (1 - \pi))}$$
 where π is the base rate

With 90% recall, a 2% false-alarm rate and a base rate of 3.4%, precision is about 61%. Move the same model to a fleet where 0.17% of machines fail on a given day and precision falls to roughly 7% — nothing about the model changed.

Figure 16 · Precision collapses when the target is rare — at unchanged model quality

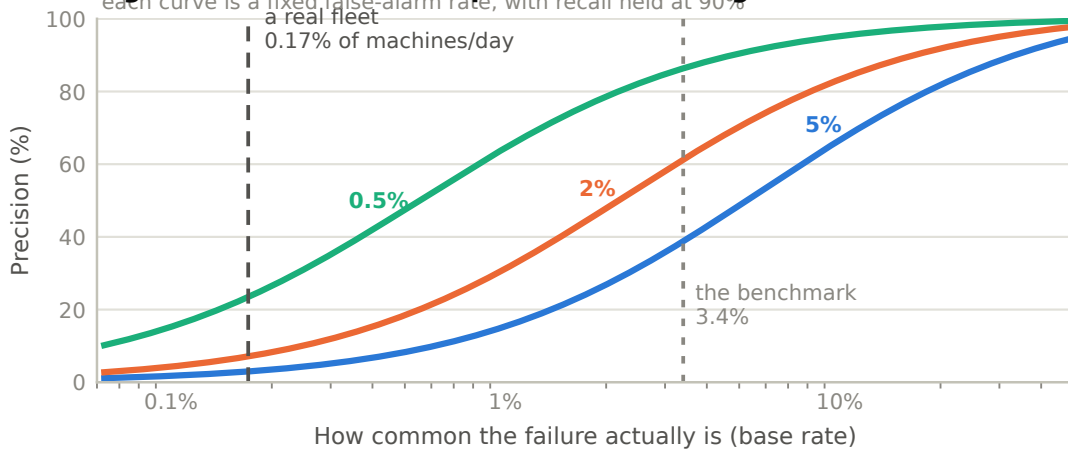


Figure 16. Precision against base rate, at three fixed false-alarm rates. The benchmark you train on and the fleet you deploy to sit at very different places on this axis, and the gap between them explains most of the disappointment in industrial predictive maintenance.

2.6 Label shift — the drift you meet on day one

When the balance of classes differs between where you trained and where you deploy, that is **label shift**. It is one of the three kinds of drift Module 2B covers, and unlike the others you usually meet it immediately, because curated benchmarks are almost always enriched with positive cases relative to reality.

The consequence is specific and worth memorising: a model's **ranking** of machines by risk still transfers to the new setting. Its **probabilities** do not, and neither does any threshold you set on them. Recompute precision at the deployment base rate before quoting a single number to the business.

2.7 Data leakage — the most expensive mistake in the pack

Leakage is when information that would not be available at prediction time has found its way into the model's inputs. The model uses it, scores brilliantly offline, and collapses in production — because in production that information does not exist yet.

The test is always a question about time: **at what moment in the real process does this value become known?** If the answer is "after the thing we are predicting", it is leakage, no matter how predictive it looks.

Figure 17 · Leakage is a question about time: when does this value become known?

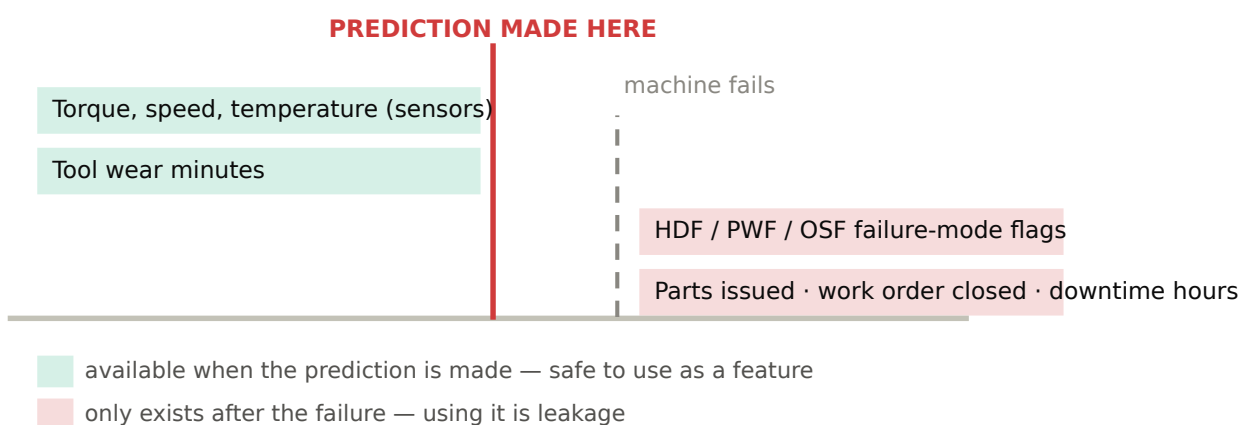


Figure 17. Sensor readings exist before the prediction is made, so they are legitimate features. Failure-mode flags, parts issued and closed work orders are all written *after* the machine failed — they are consequences, not predictors. They sit in the same database table looking exactly like features, which is what makes this trap so easy to fall into.

Type	What happens	How to prevent it
Target	An input is a consequence of the outcome	Trace every column to the moment it becomes known
Temporal	Features computed across the whole series, including the future	Compute causally — only from data available at prediction time
Group	The same machine or document appears in both train and test	Split by group, not by row
Preprocessing	Scaling or imputation fitted before splitting	Fit every transformation inside the training fold only
Tuning	The test set used repeatedly to choose settings	Iterate on validation; touch the test set once
Duplicate	Near-identical records across the split	De-duplicate, including near-duplicates, before splitting

The rule of thumb. If your offline results look surprisingly good, assume leakage until you have ruled it out. Being pleasantly surprised by a model is usually a warning, not a success.

2.8 Feature engineering — why three lines of physics beat a bigger model

A model can only use signal that is present in its inputs. If the quantity that actually governs the failure is a *combination* of the columns you have, a simple model given that combination will beat a sophisticated model that has to infer it.

Three derived features for the Task 2 data

mechanical power = torque × 2π × rpm / 60 [watts] temperature rise = process_temp - air_temp [kelvin] overstrain = tool_wear × torque [min·Nm]

Each is one line, each is computable strictly before failure, and each corresponds to a physical mechanism by which the machine actually breaks. This is what Module 3 means by *data-centric* improvement, and it is why "try a bigger model" ranks last in a well-built hypothesis backlog.

2.9 The alarm budget — the constraint that outranks the model

The final concept in Task 2 is not statistical at all. It is capacity. A prediction only has value if somebody acts on it, and people stop acting on alerts long before the alerts stop arriving.

The arithmetic, in full

predictions per day = 1,200 machines × 1 score per day = 1,200 genuine degradations/day = 1,200 × 0.17% ≈ 2 alerts anyone will read = 6 engineers × 5 each = 30 false alerts allowed = 30 - 2 = 28 maximum false-alarm rate = 28 / 1,198 ≈ 2.3% best possible precision = 2 / 30 ≈ 6.7%

Read the last line carefully. Even with a *perfect* model, at the maximum alert volume this organisation can absorb, fourteen of every fifteen alerts an engineer opens will be nothing. That is arithmetic forced by a rare event and a finite number of engineers — no amount of accuracy escapes it.

What to do when you find this. The honest deliverable is not a threshold. It is a sentence: "at the alert volume we can absorb we catch ___% of degradations; to do better we need more engineers, or a cheaper triage step that lets an alert be dismissed from a desk instead of dispatching a van." That second option is usually the right answer, and you only find it by doing this arithmetic in week one rather than month nine.

3.1 What a taxonomy has to satisfy

A taxonomy is the set of categories you sort failures into. It looks like an administrative chore and it is actually the foundation everything else rests on, because every count, every Pareto chart and every model trained afterwards inherits its quality.

Requirement	What it means in practice
Mutually exclusive	Any record belongs in exactly one category. If two categories both plausibly fit a common case, they need merging or redefining.
Collectively exhaustive	Every record has somewhere to go. Include an explicit "unclear" bucket rather than forcing records into the nearest category.
Defined by example	A one-line definition is not enough. Each category needs two or three real records, including a borderline one.
Actionable	Categories should map to different responses. If two categories always lead to the same action, they are one category.
Written before you look	Categories derived while reading the data drift toward whatever you happened to read first.

The "unclear" bucket is a diagnostic, not a failure. If more than roughly 15% of records land there, the problem is your category definitions, not the records. Report the number; never make it disappear by forcing records elsewhere.

3.2 Agreement, and why raw agreement flatters

Give the same fifty records to two experienced engineers and they will not produce identical labels. How much they agree tells you how well-defined the task actually is. But raw agreement — the percentage of records where they matched — is misleading, because some of that matching happens by luck.

Cohen's kappa fixes this by subtracting the agreement you would expect from chance alone, then expressing what is left as a fraction of the agreement that was available above chance. For more than two annotators the equivalent is **Fleiss' kappa**.

Cohen's kappa

$$\kappa = (p_{\text{observed}} - p_{\text{chance}}) / (1 - p_{\text{chance}})$$

$\kappa = 1$ means perfect agreement. $\kappa = 0$ means no better than two people guessing independently with the same category frequencies. Negative values mean systematically worse than chance, which usually indicates the two people were using different definitions entirely.

Figure 18 · Raw agreement flatters. Kappa subtracts the agreement you would get by luck.

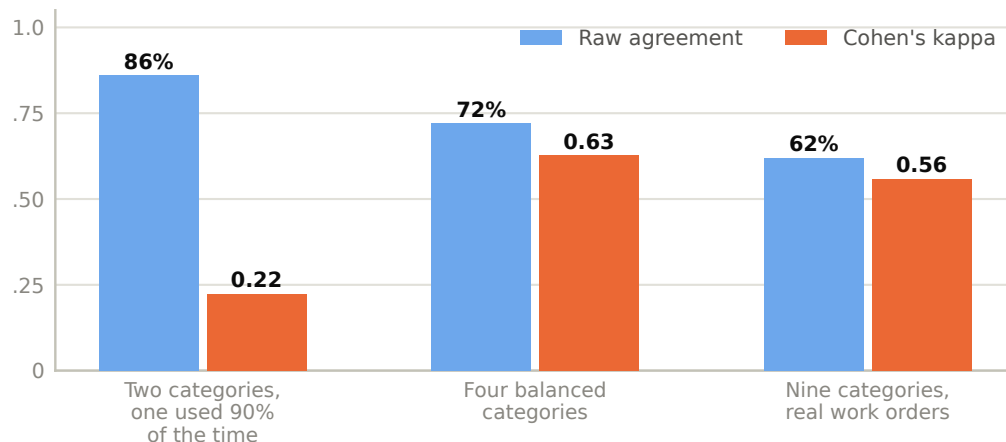


Figure 18. The first case is the trap: two categories, one used almost always, 86% raw agreement — and a kappa of 0.22, because two people who both simply guessed the common category would have agreed 82% of the time anyway. Raw agreement rewards imbalance; kappa does not.

Figure 19 · How to read a kappa — and what each band means for your project

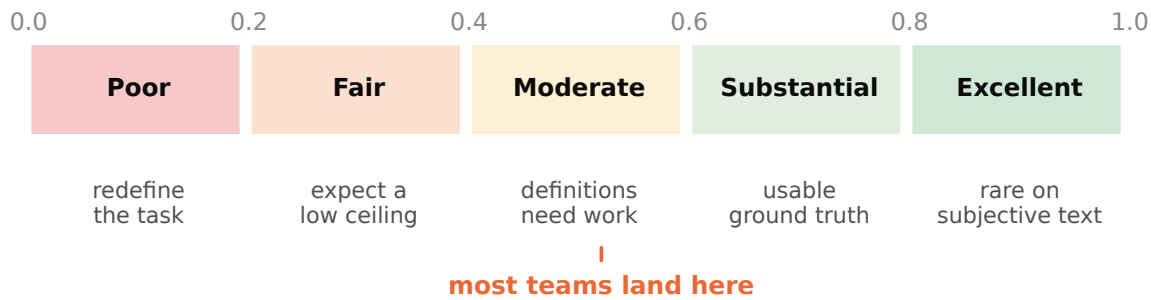


Figure 19. Conventional interpretation bands. These are orientation rather than a standard — but the practical consequences at each level are real, and they belong in your project plan rather than in a footnote.

3.3 The ceiling — why label quality caps model quality

A model learns the boundary between categories from the labels it is given. Where two qualified people disagree, there is no consistent boundary in the data — so no architecture, no amount of training data, and no amount of compute can find one.

Figure 20 · Label quality is a hard ceiling on model quality

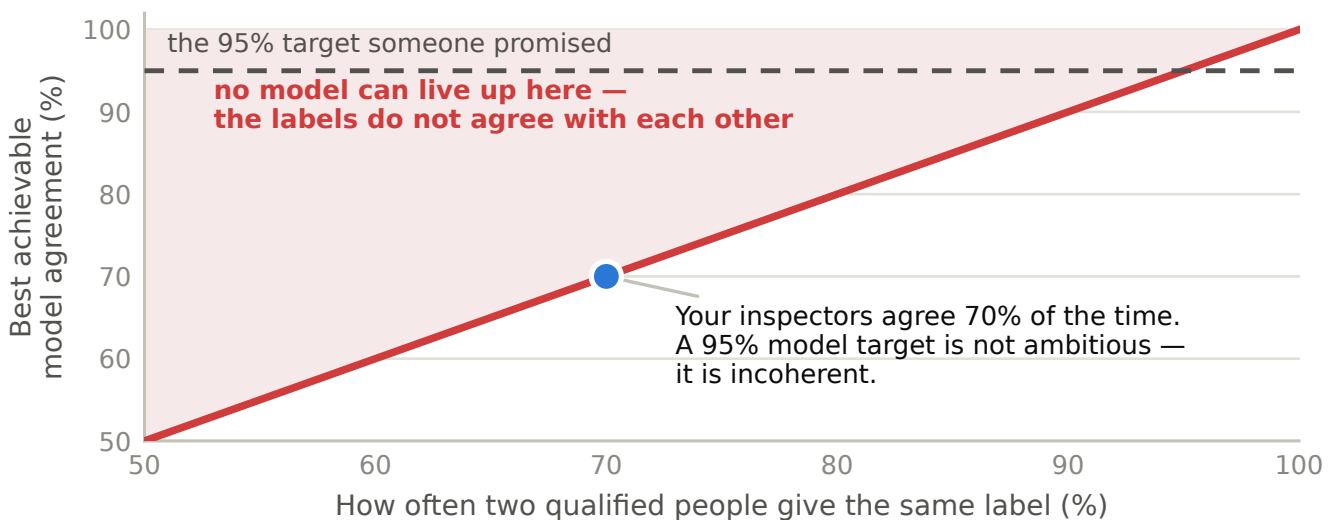


Figure 20. Your realistic ceiling is roughly the level at which humans agree with each other. If two inspectors agree 70% of the time, a 95% model target is not ambitious — it is incoherent, because it asks the model to be more consistent than the ground truth it learns from.

What to do about it. Measuring agreement is not an academic exercise — it is the cheapest way to discover that your project's stated target is impossible, and the strongest possible argument for spending a week on definitions instead of a quarter on modelling.

3.4 Pareto analysis, and why counting is not enough

The Pareto principle — that a small number of causes account for most of the effect — is reliable enough in industrial failure data to plan around. Rank the categories by frequency, plot the cumulative total, and two or three categories will usually carry most of the volume.

Then do it again weighting each category by what it costs, and you will usually get a different answer. Frequency tells you what happens most. Frequency × cost tells you where the money is, and only the second is a work plan.

Figure 21 · Same 200 work orders, two rankings. Only one is a work plan.

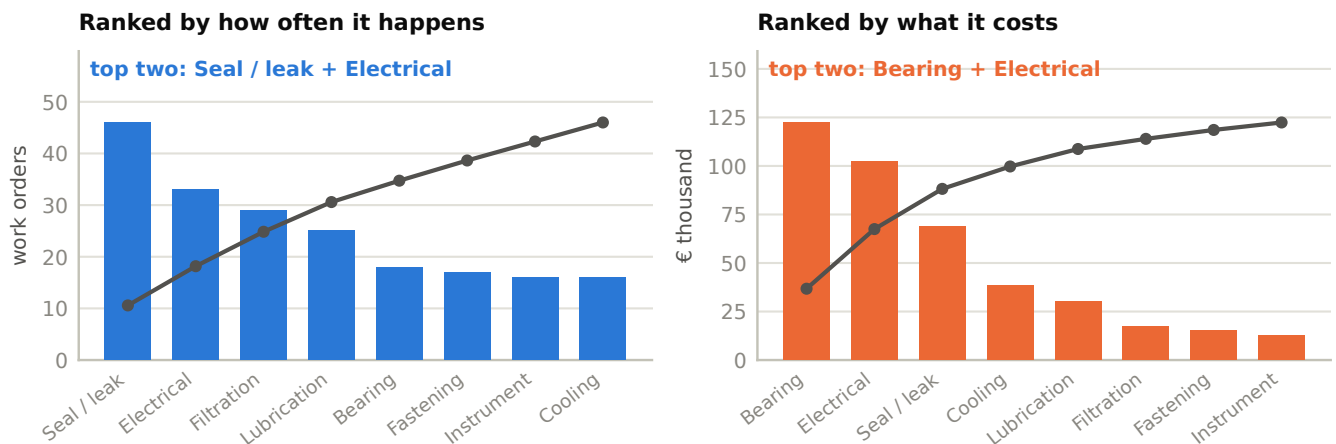


Figure 21. The same 200 work orders. Seals leak most often, but bearing failures cost several times more each, so they dominate the cost ranking despite being far less common. A backlog built from the left-hand chart sends your engineering effort to the wrong place.

3.5 Slicing — the aggregate always hides something

A single overall number averages over every kind of case. **Slice-based analysis** means recomputing the same metric separately within meaningful subsets, and it reliably surfaces problems the aggregate concealed.

Slice by	Typical finding
Failure class or defect type	Two classes carry 80% of the errors
Product family or variant	One variant sits far below the mean; the model learned the majority
Time period	Performance declined after a process change
Site, line or sensor	One plant behaves differently from all the others
Input difficulty	Recall on the hard subset is half the headline figure
Confidence band	Performance should fall smoothly with confidence; a band that misbehaves reveals miscalibration

3.6 Headroom — turning a taxonomy into a business case

Headroom is what you would gain if a category were eliminated entirely. It is deliberately optimistic — you will never eliminate a category completely — and that is fine, because its job is to rank options rather than to forecast. A category with high headroom and low effort goes first.

Headroom, and how to score a backlog

headroom = frequency × cost per occurrence × annual volume factor
priority = expected gain × confidence ÷ effort

Neither needs to be precise. They need to be **explicit**, so that ranking becomes a shared decision rather than whoever argues hardest in the room.

3.7 Using a language model as an annotator

A language model will classify two hundred work orders in a minute, in perfect JSON, with a confidence score attached to every one. This is genuinely useful and it is also where teams stop thinking.

Confidence is not agreement. The model returns clean, assured answers for exactly the records your four engineers could not agree on. Measure it the way you would measure a new human annotator — compute kappa against your human consensus — and report that, not its accuracy. An annotator that never says "unclear" on genuinely unclear text is not a good annotator. It is a fast one.

4.1 What a hypothesis is, and what it is not

"Improve retrieval quality" is an intention. A hypothesis is a claim specific enough that a measurement could contradict it. Three questions separate the two: **measured how, on what data, and what result would make you abandon it?** If you cannot answer all three, you do not have a hypothesis yet.

Level	Question it answers	Example
Business	What outcome improves?	Engineers are dispatched to fewer machines that turn out to be healthy
Technical	What change causes it?	Adding derived power and temperature-rise features improves failure ranking
Measurable	What number moves, by how much, measured how?	Recall at a 2.3% false-alarm rate, on the held-out 3,000 rows, rises from 0.48 to at least 0.60

The **null hypothesis** (H_0) is the assumption that your change did nothing. You never prove your claim directly — you assume the null and ask whether the evidence is strong enough to reject it. Failing to reject is not the same as showing there is no effect; it means you have not shown there is one.

The courtroom analogy. A court does not set out to prove innocence. It assumes it, and asks whether the evidence is strong enough to overturn that assumption. If not, the verdict is "not proven" — not "proven innocent". Statistical testing works exactly this way, and the distinction matters when someone asks you whether a change "made no difference".

4.2 What a p-value actually says

A **p-value** is the probability of seeing a difference at least as large as the one you observed, *if your change had done nothing at all*. Small p means chance alone rarely produces something this big, so the null explanation is uncomfortable.

Figure 22 · A p-value answers one question: how often would chance alone do this?

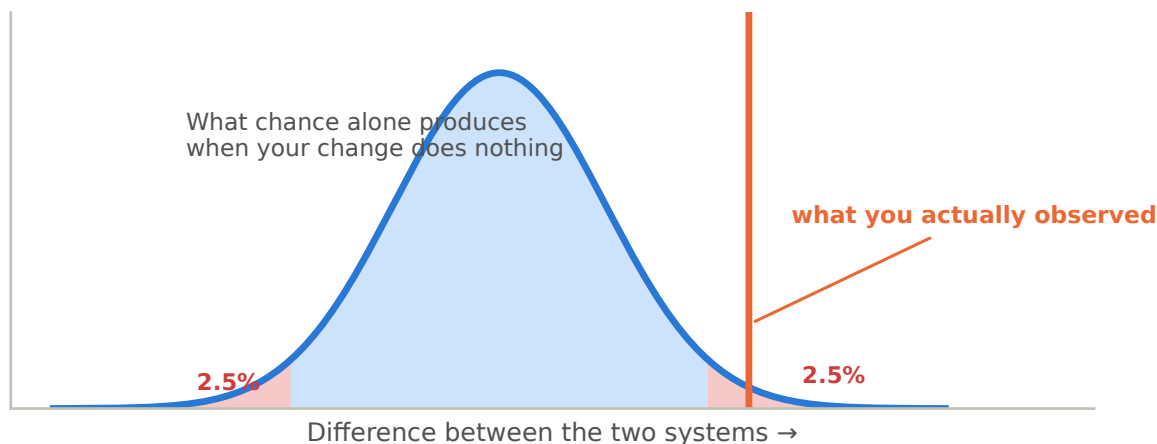


Figure 22. The curve is what chance alone produces when your change does nothing. Your observed result is the orange line. The p-value is the proportion of that curve lying at least as far out as your result — nothing more, and nothing about how large or important the effect is.

Three things a p-value is not. It is not the probability that your change worked. It is not the probability the null is true. And it says nothing whatever about whether the effect is big enough to matter — for that you need the effect size, in section 4.7. A p-value of 0.001 on a 0.2-point improvement means you have very reliably detected something not worth having.

4.3 Paired versus unpaired — the design choice that decides everything

When you compare two systems on the same evaluation set, the items differ enormously in difficulty. Some are hard and both systems fail; some are easy and both succeed. An **unpaired** comparison treats all of that variation as noise — and the noise is

usually larger than the improvement you are hunting.

A **paired** comparison looks at each item individually and asks how the two systems did on *that* item. Item difficulty cancels, and much smaller real effects become detectable with the same amount of data.

Figure 23 · Pairing is not a refinement — it is the difference between seeing an effect and missing it

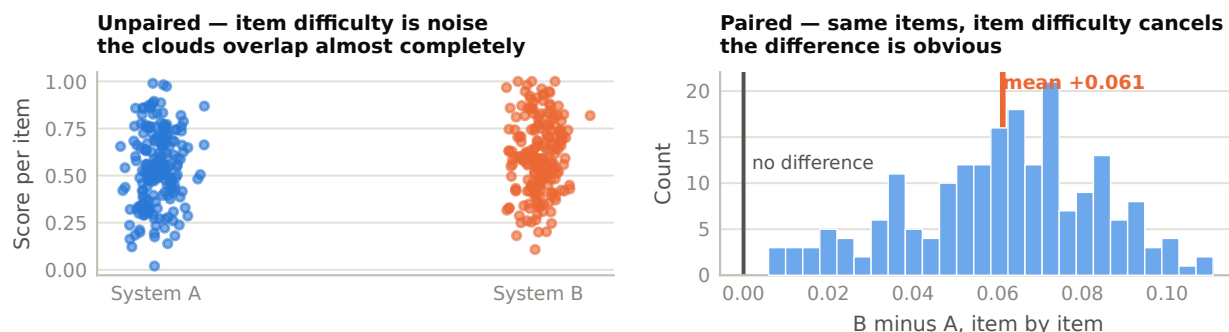


Figure 23. The same data both times. On the left, two clouds of scores that overlap almost entirely — an unpaired test sees nothing. On the right, the per-item differences, which sit almost entirely above zero. The effect was always there; the unpaired design simply could not see it.

4.4 McNemar's test — the right test for two models on the same items

When both systems are scored on the same items and each item is simply right or wrong, McNemar's test is the correct choice — and that is the situation almost every time you compare two models.

Figure 24 · McNemar's test — the only two cells that carry information

	System B RIGHT	System B WRONG	
System A RIGHT	a = 271 both right <i>tells you nothing</i>	b = 14 A right, B wrong <i>counts</i>	Only b and c matter. McNemar asks whether 14 and 33 differ by more than chance allows. If $b + c$ is small, the two systems barely disagree — no test will find a difference, and that is your answer.
System A WRONG	c = 33 A wrong, B right <i>counts</i>	d = 82 both wrong <i>tells you nothing</i>	

Figure 24. Items where both systems agreed carry no information about which is better, however many there are. Only the two discordant cells matter, and the test asks whether they differ by more than chance allows.

A practical point worth making in your readout. If $b + c$ is small — the two systems rarely disagree — then no test will find a difference, and that is itself the answer. Your change did very little. You have learned something useful without needing a p-value at all.

4.5 Permutation tests — when you trust no assumption

Every named test carries assumptions about how the data is distributed. A **permutation test** carries none. It builds the "nothing happened" world directly from your own data, by repeatedly shuffling which result belongs to which system and seeing how often chance alone produces a difference as large as yours.

The permutation test, in five steps

1. Compute the observed difference between the two systems
2. Pool the results and randomly reassign each item's outcome between them
3. Recompute the difference on that shuffled assignment
4. Repeat 10,000 times to build the null distribution
5. p = the proportion of shuffled differences at least as extreme as yours

Figure 25 · The permutation test: build the "nothing happened" world yourself

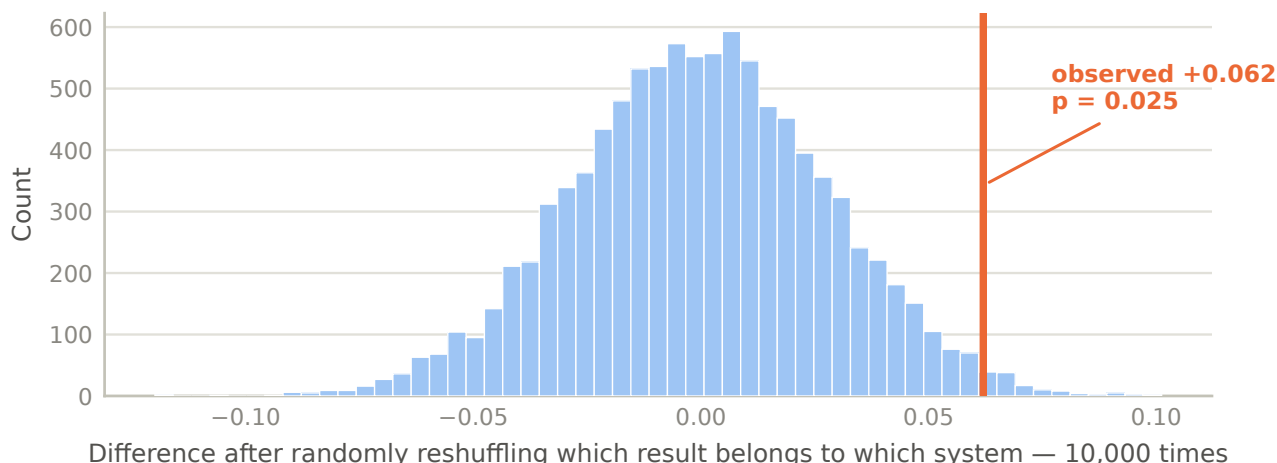


Figure 25. Ten thousand shuffles. The histogram is what chance alone produces; the orange line is what you actually got. It works for any metric — including ones with no formula at all — and it is easy to explain to a sceptical stakeholder, because you are simply asking how often coincidence would do this.

4.6 The two ways to be wrong, and statistical power

Error	You concluded	Consequence
Type I (α)	The change helped — when it did not	You ship complexity that costs money for no benefit, and you trust a method that does not work. Controlled by the significance level, conventionally 0.05.
Type II (β)	The change did not help — when it did	You discard a real improvement, usually because the evaluation set was too small. Controlled by power, conventionally targeted at 0.80.

Figure 26 · The two ways to be wrong — and power is $1 - \beta$

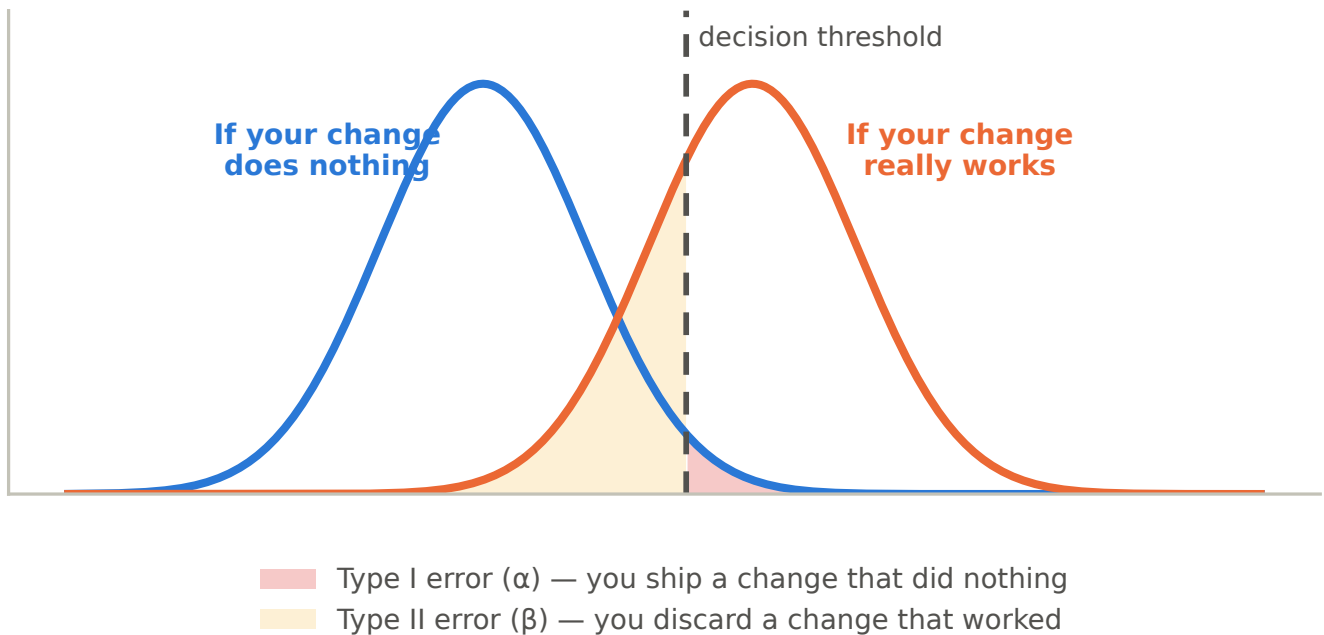


Figure 26. Two worlds, one decision threshold. The red area is how often you declare success when your change did nothing. The amber area is how often you miss a change that genuinely worked. Moving the threshold trades one for the other; only more data shrinks both.

Power is $1 - \beta$: the probability that your experiment detects an effect that is genuinely there. Four quantities are linked — significance level, power, effect size and sample size — and fixing any three determines the fourth. In practice you fix the first three and solve for how much data you need.

Figure 27 · Halving the effect you want to see roughly quadruples the data you need

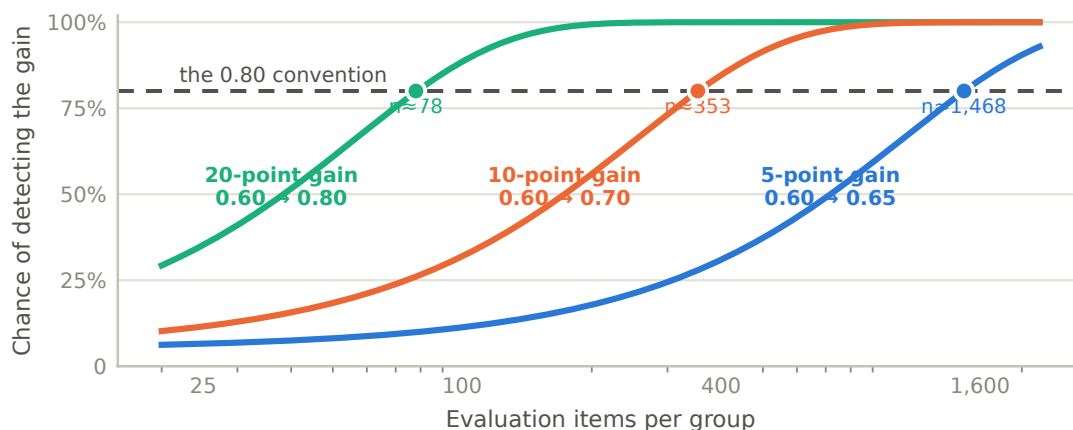


Figure 27. To detect a 20-point improvement with 80% power you need about 80 items per group. For 10 points, about 350. For 5 points, about 1,500. Halving the effect you want to see roughly quadruples the data required — which is why an evaluation set of thirty examples can only ever confirm large, obvious wins.

Do the power calculation before the experiment, not after. Most teams discover their planned evaluation set cannot detect the effect they are hoping for. That discovery is uncomfortable and it is the most valuable output of the exercise — it is far cheaper to learn it now than to spend three weeks producing an inconclusive result.

4.7 Effect size — is it big enough to care about?

Significance tells you the measurement is reliable. **Effect size** tells you whether what it measured is worth having. They are different questions and you need both.

Measure	What it expresses	How to read it
Absolute difference	The raw change in the metric	The most honest for a business conversation: 0.61 → 0.75 is +14 points
Relative difference	The change as a percentage of the baseline	+23% — persuasive, and it inflates small absolute changes. Never quote it alone.
Cohen's d	The difference in standard deviations	0.2 small, 0.5 medium, 0.8 large
Number needed	How many items for one extra success	The version an operations manager will actually use

The weighing-scale analogy. Statistical significance tells you the scale is precise enough to trust the reading. Effect size tells you whether the difference it shows is worth caring about. A scale precise enough to detect two grams in a shipment of steel is working perfectly and telling you nothing useful.

4.8 Multiple comparisons, and the fix that costs nothing

Test one idea at the 0.05 threshold and you have a one-in-twenty chance of a false positive. Test twenty ideas and you should *expect* one to look significant even if none of them work.

Figure 28 · Test twenty ideas and expect one to look real when none of them are

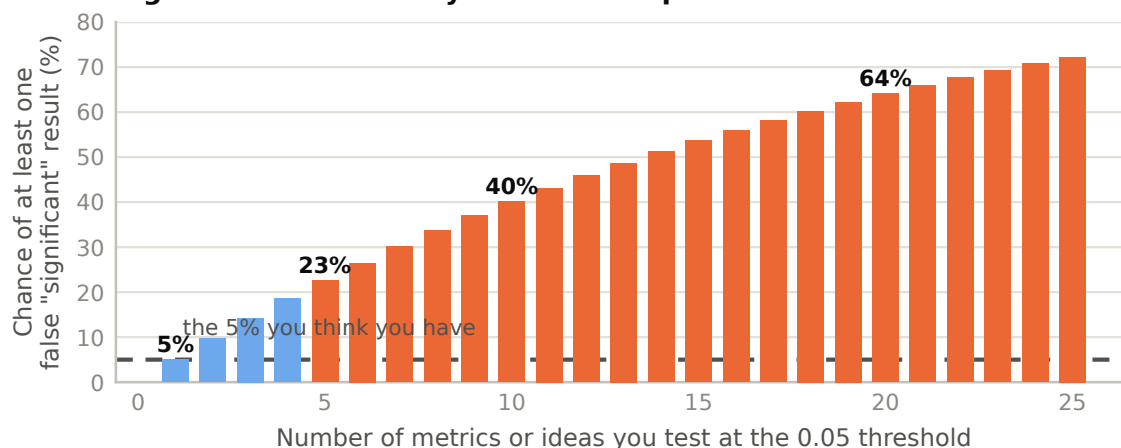


Figure 28. The probability of at least one false "significant" result, against the number of things you test. At five tests it is already 23%. This is how a team evaluates a change against fifteen metrics, finds two that improved, and reports those two — with nothing dishonest intended and a result that is close to meaningless.

Correction	How it works	Trade-off
Bonferroni	Divide the threshold by the number of tests	Very conservative; loses power quickly
Holm–Bonferroni	Sequentially rejective version	Uniformly more powerful; a good default
Benjamini–Hochberg	Controls the expected proportion of false discoveries	Best when screening many candidates
Pre-registration	Name the primary metric and pass criterion <i>before</i> running	Not a correction — a discipline. The strongest defence, and it is free.

4.9 Peeking — why checking as you go is not neutral

Checking results repeatedly and stopping the moment they look favourable dramatically inflates false positives, because every look is another chance for noise to cross the line.

Figure 29 · Peeking — checking until it looks good is not a neutral act

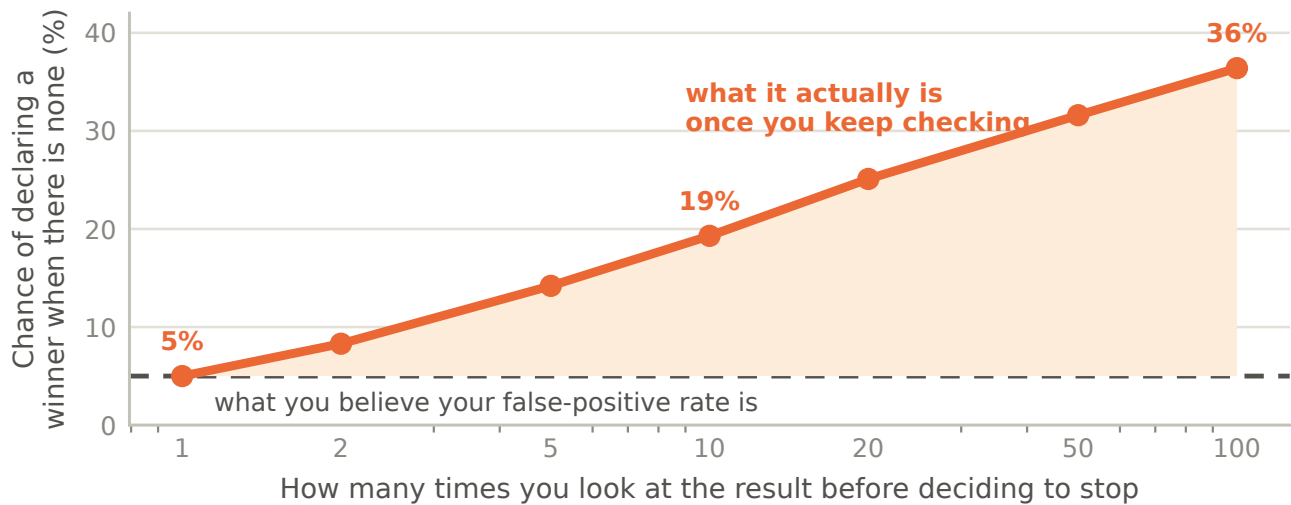


Figure 29. Two identical systems, evaluated repeatedly as items accumulate, stopping the first time p drops below 0.05. With enough looks, the chance of declaring a winner where none exists climbs past a third. The fix is to fix the sample size and stopping rule in advance, or to use a sequential design that accounts for planned interim looks.

The same problem in offline evaluation. Running your change against the gold set, adjusting, re-running, adjusting again and reporting the best result is peeking under another name. Each pass consumes a little of the test set's honesty. This is precisely why the test set is held out and the validation set is where you iterate.

4.10 Calibration — do the model's numbers mean anything?

A model that outputs 0.80 is making a claim: that things it labels 0.80 happen about 80% of the time. **Calibration** is whether that claim is true. It is a separate property from accuracy — a model can rank cases perfectly and still be badly calibrated.

Figure 30 · Calibration — do the numbers your model outputs mean anything?

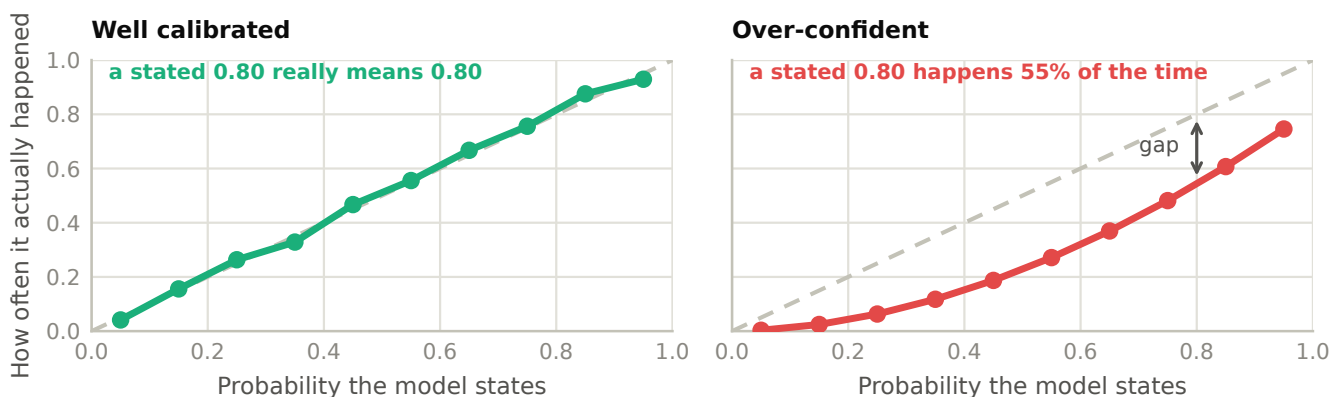


Figure 30. A reliability diagram: bin the predictions by stated probability, and plot what actually happened in each bin against the diagonal. The right-hand model ranks cases just as well as the left-hand one, but its stated 0.80 corresponds to a real rate near 0.55 — so every threshold rule built on it will behave unexpectedly.

Brier score — calibration in one number

$\text{Brier} = \text{mean}((\text{predicted probability} - \text{actual outcome})^2)$ lower is better

Report the reliability curve alongside it. The single number tells you whether there is a problem; the curve tells you where it is, which is what you need in order to fix it.

4.11 Drift — three kinds, three responses

Models degrade because the world moves after training. The three ways it moves are genuinely different and they demand different fixes, so naming the right one matters.

Figure 31 · Three kinds of drift — three different responses

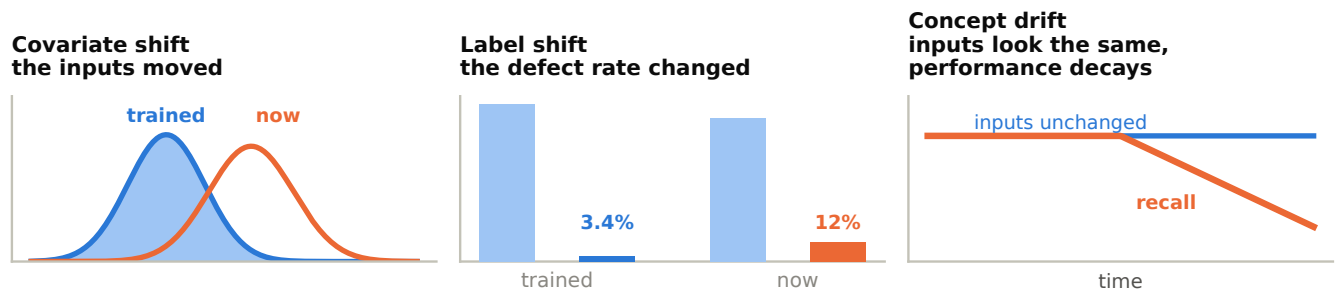


Figure 31. Covariate shift: the inputs moved — new variants, new lighting, a new sensor. Label shift: the balance of outcomes moved — the defect rate rose. Concept drift: the inputs look the same and the relationship between input and outcome has changed underneath, so performance decays while nothing visible has moved.

Type	How to detect it	What to do
Covariate shift	Compare input feature distributions over time — KS test, PSI	Retraining on recent data is often enough
Label shift	Monitor predicted class proportions	Recalibrate thresholds; check the process, not just the model
Concept drift	Performance falls while inputs look unchanged	Retraining required; old labels may no longer apply

Two detectors you will implement in Task 4

Kolmogorov–Smirnov test – largest gap between two cumulative distributions. A small p-value means the feature's distribution moved. Population Stability Index – $\sum (\text{recent\%} - \text{reference\%}) \cdot \ln(\text{recent\%} / \text{reference\%})$ PSI < 0.10 stable no action 0.10 – 0.25 moderate watch it PSI > 0.25 significant investigate and consider retraining

Why covariate shift matters operationally. It is the only kind detectable *without labels*. In production, labels arrive late — sometimes weeks late, sometimes never. Input monitoring gives you a signal the same day. It will not tell you performance dropped; it tells you to go and check.

Spread and sampling

σ standard deviation – typical distance from the average $\sigma / \sqrt{n}$ standard error of a mean of n items $\pm 1\sigma$ / $\pm 2\sigma$ / $\pm 3\sigma$ contain 68.3% / 95.4% / 99.7% of a normal distribution

Control charts (subgroup size $n = 5$)

$\bar{X}$ chart limits = $\bar{X} \pm A_2 \cdot \bar{R}$ $A_2 = 0.577$ R chart limits = $D_3 \cdot \bar{R}$ to $D_4 \cdot \bar{R}$ $D_3 = 0$, $D_4 = 2.114$ σ from ranges = $\bar{R} / d_2$ $d_2 = 2.326$ ARL_0 = mean subgroups between false alarms (3σ rule alone ≈ 370)

Capability

$C_p = (USL - LSL) / 6\sigma$ $C_{pk} = \min(USL - \mu, \mu - LSL) / 3\sigma$ 95% CI half-width $\approx 1.96 \cdot \sqrt{1/(9n) + C_{pk}^2 / (2(n-1))}$

Classification

Recall = $TP / (TP + FN)$ Precision = $TP / (TP + FP)$ FPR = $FP / (FP + TN)$ F1 = $2PR / (P + R)$
Precision at a base rate π : $TPR \cdot \pi / (TPR \cdot \pi + FPR \cdot (1 - \pi))$ Expected cost = $FP \cdot \text{cost}(FP) + FN \cdot \text{cost}(FN) + TP \cdot \text{cost}(TP) + TN \cdot \text{cost}(TN)$

Agreement

$\kappa = (p_{\text{observed}} - p_{\text{chance}}) / (1 - p_{\text{chance}})$ Cohen's κ – two annotators. Fleiss' κ – three or more. Model ceiling $\approx$ the level at which qualified humans agree with each other.

Comparison and evidence

McNemar uses only b and c , the items where exactly one system was right Permutation p = proportion of 10,000 shuffles at least as extreme as observed Bootstrap resample n from n with replacement, 2,000–10,000 times, take the 2.5th and 97.5th percentiles Paired bootstrap – resample once, score both systems, take the difference, then check whether the interval on the difference contains zero $P(\text{at least one false positive in } k \text{ tests}) = 1 - 0.95^k$

Calibration and drift

Brier = $\text{mean}((\text{predicted probability} - \text{outcome})^2)$ PSI = $\sum (\text{recent\%} - \text{reference\%}) \cdot \ln(\text{recent\%} / \text{reference\%})$ < 0.10 stable $\cdot 0.10$ – 0.25 moderate $\cdot > 0.25$ significant

Prioritisation

headroom = frequency $\times$ cost per occurrence $\times$ annual volume priority = expected gain $\times$ confidence $\div$ effort

Term	Plain meaning
Aleatoric uncertainty	Irreducible noise. Two genuinely identical inputs give different outcomes. More data does not help.
ARL_0 / ARL_1	Average subgroups between false alarms while healthy / to detect a real fault.
Base rate (prior)	How common the thing you are hunting actually is, in the place you will deploy.
Bootstrap	Resampling your own data with replacement to measure how uncertain a number is.
Brier score	Mean squared error of predicted probabilities. Lower is better calibrated.
Calibration	Whether a stated probability matches the observed frequency.
Common cause	The process breathing. Always present, no single culprit.
Concept drift	The input-to-outcome relationship changed while inputs look unchanged.
Confidence interval	The range of values consistent with the data you have.
Covariate shift	The input distribution moved. The only drift detectable without labels.
C_p / C_{pk}	Process capability: spread against tolerance, and the same penalised for being off centre.
Effect size	How big the difference is, as opposed to how reliably you detected it.
Epistemic uncertainty	The model does not know because it has not seen enough of this case. Targeted data helps.
Fleiss' / Cohen's kappa	Agreement between annotators, corrected for agreement that would happen by chance.
Headroom	What you would gain if a failure category were eliminated entirely.
Label shift	The balance of outcomes differs between training and deployment.
Leakage	Information in the model's inputs that would not exist at prediction time.
McNemar's test	The correct test when two systems are scored right/wrong on the same items.
Null hypothesis (H_0)	The assumption that your change did nothing. You reject it or fail to.
Peeking	Checking repeatedly and stopping when results look good. Inflates false positives sharply.
Permutation test	Building the "nothing happened" distribution by shuffling your own data.
Power	The chance your experiment detects an effect that is genuinely there.
Precision	Of the alerts you raised, how many were real.
Pre-registration	Naming the primary metric and pass criterion before running the experiment.
PSI	Population Stability Index — how far a feature's distribution has moved.
Recall (sensitivity)	Of the events that happened, how many you caught.
ROC curve	Recall against false positive rate across all thresholds. Flattering on rare events.
Special cause	Something specific went wrong. Has a findable cause worth finding.
Standard error	How much a sample average would bounce around if you repeated the sampling.
Threshold	The score above which you act. A choice you make, not something the model learned.
Type I / Type II error	Shipping a change that did nothing / discarding a change that worked.

If you remember five things. A single accuracy number is not a specification. Every headline figure needs an interval. State the hypothesis and the pass criterion before the experiment, and check you have the power to detect it. Find the failure pattern before reaching for more data. And threshold and calibration work is nearly free — it is often the largest single gain available.